Lab 02: Data Validation & Cleaning

Code Explanation Guide

DSC3103 - Data Science (Systems & Technologies)

Complete breakdown of all code files, their purpose, and how they work together

Table of Contents

1. Project File Structure

Your project is organized as follows:

```
dsc3103-project/
|
|-- data/
|   |-- raw/
|   |   |-- prices.csv          <-- THE RAW, MESSY DATA (never touch!)
|   |
|   |-- processed/
|   |   |-- prices_clean.parquet  <-- CLEANED OUTPUT
|   |   |-- cleaning_log.md       <-- LOG OF ALL CHANGES
|
|-- src/
|   |
|   |-- validate/
|   |   |-- rules.py           <-- VALIDATION RULES (what to check)
|   |   |-- profile.py         <-- DATA PROFILER (understand data)
|   |
|   |-- transform/
|   |   |-- clean.py           <-- DATA CLEANER (fix problems)
|
|-- docs/
|   |-- validation_report.md   <-- SUMMARY REPORT
|   |-- lab_02_notes.md        <-- REFLECTION NOTES
|   |-- price_histogram.png   <-- VISUALIZATION
|
|-- generate_messy_prices.py  <-- CREATES THE MESSY DATA
|-- check.py                  <-- TESTS ALL RULES
```

2. The Big Picture: How Everything Connects

Think of this as a factory assembly line for data:

Step 1: GENERATE (generate_messy_prices.py)

Creates messy data with intentional errors. Like a factory making defective products on purpose.

Step 2: TEST (check.py)

Runs validation rules to see what problems exist. Like an inspector finding issues.

Step 3: PROFILE (profile.py)

Analyzes the data thoroughly. Like a scientist studying the data's characteristics.

Step 4: CLEAN (clean.py)

Fixes the problems found by rules. Like a worker repairing defective products.

Step 5: VERIFY

Ensures raw data was never touched. Like quality control checking the original is safe.

3. File-by-File Explanation

3.1 generate_messy_prices.py - Creating Messy Data

PURPOSE: Creates a CSV file with intentional data quality problems for practice.

WHAT IT GENERATES:

- Negative prices (like -10.50)
- Duplicate record_ids (same ID appears twice)
- Duplicate rows (exact copies of other rows)
- Invalid dates (like "2020-14-50" which is impossible)
- Missing market values (empty strings)
- Inconsistent commodity casing ("MAIZE", "maize", "Maize")

SIMPLIFIED CODE:

```
# Key parts simplified:

# 1. Define valid options
COMMODITIES = ["Maize", "Beans", "Rice", "Wheat", "Coffee", "Tea"]
MARKETS = ["Central Market", "East Side Market", ...]

# 2. Sometimes make casing inconsistent
if random.random() < 0.1:  # 10% chance
    commodity = commodity.upper()  # "Maize" -> "MAIZE"

# 3. Sometimes make negative prices
if random.random() < 0.03:  # 3% chance
    price = -abs(random.uniform(1, 50))  # -5.23, -12.87, etc.

# 4. Sometimes create duplicate IDs
if random.random() < 0.02:  # 2% chance
    dup_id = random.choice(rows)["record_id"]  # Copy another row's ID

# 5. Write to CSV
with open("data/raw/prices.csv", 'w') as f:
    writer = csv.DictWriter(f, fieldnames=[...])
    writer.writeheader()
    writer.writerows(rows)
```

WHY THIS CODE:

Data scientists need to practice cleaning messy real-world data. This generator simulates common data problems you'll encounter: negative values that should be positive, duplicate IDs that inflate counts, inconsistent text formatting, and impossible dates.

3.2 rules.py - The Validation Rules

PURPOSE: Defines WHAT problems to look for in the data. Rules only DETECT, never fix.

THE 6 RULES:

RULE | WHAT IT CHECKS | RETURNS

rule_positive_price | Price < 0 | Rows with negative prices

rule_duplicate_ids | Same record_id twice | Rows with duplicate IDs

rule_duplicate_rows | Entire row is identical | Rows that are exact copies

rule_valid_date | Date is YYYY-MM-DD | Rows with invalid dates

rule_missing_market | Market is empty/NaN | Rows missing market

rule_known_commodity | Casing matches 'Maize' | Rows with wrong casing

SIMPLIFIED CODE - Positive Price Rule:

```
# Original (complex):
def rule_positive_price(df):
    df_temp = df.copy()
    df_temp["price_numeric"] = pd.to_numeric(df_temp["price"], errors="coerce")
    neg_prices = df_temp[df_temp["price_numeric"] < 0].copy()
    neg_prices["Reason"] = "Negative Price"
    return neg_prices[["record_id", "price", "Reason"]]

# Simplified logic:
# 1. Make a copy of the data
# 2. Convert price column to numbers (letters become NaN)
# 3. Find rows where price < 0
# 4. Add a "Reason" column explaining why
# 5. Return only the relevant columns
```

SIMPLIFIED CODE - Date Validation:

```
def rule_valid_date(df):
    def is_valid_date(date_str):
        # Try to parse as YYYY-MM-DD
        try:
            datetime.strptime(str(date_str), "%Y-%m-%d")
            return True   # Success = valid
        except ValueError:
            return False  # Failed = invalid

    # Keep rows where date IS valid (the ~ flips it)
    invalid_dates = df[~df["date"].apply(is_valid_date)]
    return invalid_dates
```

WHY THIS CODE:

Validation rules are separate from cleaning because they serve different purposes. Rules identify problems; cleaning fixes them. This separation lets you run the same checks at different stages to measure improvement.

3.3 check.py - Testing the Rules

PURPOSE: Runs every validation rule against the data to see what problems exist.

SIMPLIFIED CODE:

```
# 1. Import all the rules
from src.validate.rules import (
    rule_positive_price,
    rule_duplicate_ids,
    rule_duplicate_rows,
    rule_valid_date,
    rule_missing_market,
    rule_known_commodity
)

# 2. Load the messy data
df = pd.read_csv("data/raw/prices.csv")

# 3. Call each rule and print results
neg_prices = rule_positive_price(df)
print(f"Found: {len(neg_prices)} rows with negative prices")

dup_ids = rule_duplicate_ids(df)
print(f"Found: {len(dup_ids)} rows with duplicate IDs")

# ... repeat for all 6 rules
```

WHY THIS CODE:

Before cleaning, you need to know what problems exist. check.py acts as a diagnostic tool - it tells you exactly what's wrong so you can decide how to fix each problem.

3.4 profile.py - Understanding the Data

PURPOSE: Creates a comprehensive statistical report of the data.

WHAT IT REPORTS:

1. Schema - Column names and data types
2. Row count - How many records exist
3. Missing values - Empty cells per column (with percentages)
4. Duplicate analysis - Both ID duplicates and exact duplicates
5. Invalid values - Count of problems found by each rule
6. Summary statistics - Mean, median, min, max, std for numbers
7. Value counts - How many times each commodity/market appears
8. Visualization - Price histogram saved as PNG

KEY CODE PATTERN:

```
# Summary statistics with describe()
df["price_numeric"] = pd.to_numeric(df["price"], errors="coerce")
stats = df[["price_numeric", "quantity_numeric"]].describe()
# Returns: count, mean, std, min, 25%, 50%, 75%, max

# Missing value percentage
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)

# Histogram visualization
df["price_numeric"].dropna().hist(bins=50, ax=ax)
plt.savefig("docs/price_histogram.png")
```

WHY THIS CODE:

Profiling helps you understand the data's shape before deciding how to clean it. The histogram, for example, helps you decide what counts as an outlier vs a normal extreme value.

3.5 clean.py - Cleaning the Data

PURPOSE: Fixes the problems identified by rules, logging every decision.

THE 3 DECISION TYPES:

REJECT = Remove the row entirely (can't be fixed)

IMPUTE = Replace missing value with a reasonable guess

NORMALIZE = Fix formatting without losing information

DECISIONS MADE FOR EACH RULE:

PROBLEM | ACTION | WHY

Duplicate rows | REJECT | Copies add no new information

Duplicate IDs | REJECT | Keep first occurrence only

Invalid dates | REJECT | Can't guess the correct date

Negative prices | REJECT | Prices can't be negative

Missing market | REJECT | Market is essential, can't impute

Bad casing | NORMALIZE | 'MAIZE' -> 'Maize' preserves data

SIMPLIFIED CODE - The Cleaning Logic:

```
# HASH VERIFICATION (Step 6 - preserve raw data)
hash_before = get_file_hash("data/raw/prices.csv")

# Load data
df = pd.read_csv("data/raw/prices.csv")

# For each problem type:
if len(rule_duplicate_rows(df)) > 0:
    df = df.drop_duplicates(keep="first")
    log.append("Removed duplicate rows")

if len(rule_negative_prices(df)) > 0:
    df = df[df["price"] >= 0]
    log.append("Removed negative prices")

# Normalize casing
df["commodity"] = df["commodity"].str.title()

# Save cleaned data
df.to_parquet("data/processed/prices_clean.parquet")

# Verify raw file unchanged
hash_after = get_file_hash("data/raw/prices.csv")
if hash_before == hash_after:
    print("Raw file verified untouched!")
```

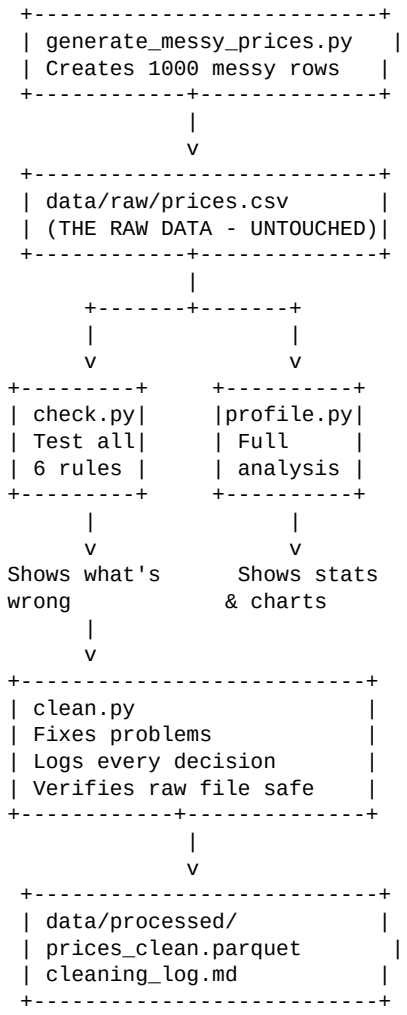
HASH VERIFICATION:

Before cleaning starts, we record the MD5 hash of the raw file. After cleaning, we hash it again. If the hashes match, we proved the original was never touched - essential for data integrity.

WHY THIS CODE:

clean.py follows the principle of logging every decision. You can always trace back what was changed and why. The hash verification proves the raw data is sacred and was never modified.

4. Data Flow Diagram



5. Key Concepts Explained

VALIDATION vs CLEANING

Validation asks 'What's wrong?' without changing data. Cleaning asks 'How do I fix it?' and modifies data. They are separate steps because you might validate multiple times (before and after cleaning) to measure improvement.

REJECT vs NORMALIZE vs IMPUTE

REJECT: Remove bad rows entirely. Used when data is too corrupted to fix.

IMPUTE: Fill in missing values with reasonable guesses (like using the median price).

NORMALIZE: Fix formatting while keeping the information (like 'MAIZE' -> 'Maize').

WHY PARQUET FORMAT?

Parquet is a compressed, columnar storage format that's much faster to read than CSV. It's ideal for cleaned data that will be analyzed repeatedly.

HASH VERIFICATION

An MD5 hash is like a digital fingerprint. If even ONE character changes in prices.csv, the hash will be completely different. By comparing hashes before and after cleaning, we prove the original file was never touched.

DATA LEAKAGE

Leakage happens when information from the future 'leaks' into your training data. For example, if you remove errors only from your test set (not training set), your model will seem artificially accurate. The raw data must stay untouched so any dataset can be re-cleaned consistently.

--- END OF DOCUMENT ---